Document 13 — Testing, Evaluation & Observability

> This is for self-study and live mentoring. Full plan: CURRICULUM.md

Prerequisites

12_production_engineering

How to Read & Practice This Document

- **What:** proving your system works, and catching it when it stops working.
- **Why:** systems that aren't always the same break silently without this — you won't notice something got worse until a user does, and by then it's a real problem, not just a code review comment.
- **When:** before you'd ever trust a prompt or model change in production; really, before you'd trust your own Project 4 as "finished."
- **How to practice:** 1. Read the OpenAI Evals README and one example test before writing your own — copy the *shape* of it, not the content. 2. Do the **Basic/Intermediate** tests closed-book where you can — these are normal pytest, no LLM involved. 3. Do the **Real-world** LLM-as-judge exercise, writing your scoring rules before you look at any output. 4. Do the **Edge case/Failure** exercises for real — make a prompt worse on purpose and confirm your test suite actually catches it. If it doesn't, that's the real lesson. 5. Use **Hint 1** → **Hint 2** only after really trying. Ask for the full solution only if you say **"Show me the solution."** 6. Before moving on, show a real score drop from a real, on-purpose problem you caused. If you haven't made one happen, you're not done.

Jump to: Core Concepts · Practice Exercises · Build Task

The Story — what this document is actually building

By this point, Project 4 (your multi-agent system) runs. But "runs" and "works" aren't the same thing — an LLM system can run without errors while quietly giving worse answers than it did last week, and nothing on your screen will tell you.

First, you need a way to check correctness that doesn't assume the model says the exact same words every time. Two answers can use completely different wording and both be right — so instead of checking "is this exactly what I expected," you check "does this have the *properties* a correct answer must have." That's the mindset shift the **testing layers** are built on: plain, exact tests for the parts that don't touch a model at all (chunking, tool logic, routing), and property-based checks for the parts that do.

Second, once you can check correctness once, you need to check it *again and again* — every time you touch a prompt or swap a model. This is **regression testing**: a fixed set of tasks with fixed pass/fail rules, run automatically, so a change that quietly makes things worse gets caught by your test suite instead of by a user.

Third, some of what makes an answer "good" can't be checked by ordinary code at all — is this summary actually good, not just non-empty? That's what **LLM-as-a-judge** is for: a second model call, scoring the first model's answer against rules you wrote down yourself. The judge is not a neutral referee — it's a piece of software you built, with a prompt you have to get right, exactly like any other prompt.

Fourth, once a system is live, you can't watch every run happen — so you need a **record** of what happened in each one: the input, every tool call, the final output, timing, cost. Without that record, a bad answer is just a mystery you can't go back and investigate.

That's the whole story: testing layers give you a way to check correctness without assuming exact wording. A judge extends that check into "quality," not just "shape." A regression suite makes sure a change never quietly makes things worse without you noticing. And run records let you understand, after the fact, exactly what happened when something goes wrong. The **Build Task** below asks you to put the first three of these together into one real, runnable test suite for Project 4 — the thing that actually tells you whether you can trust a change before you ship it.

Core Concepts (read this first — everything you need is here)

Topics on this page: Why testing something that isn't always the same needs a different mindset · The testing layers, and what each one actually checks · Regression testing for prompts · Test sets and LLM-as-a-judge · Watching your system: what to record for each run

Why testing something that isn't always the same needs a different mindset

Normal unit tests check exact equality: given this input, the output must be exactly that. An LLM given the same input can word its answer differently each time, and still be just as correct. If your test checks for the exact same words, it fails for the wrong reason: the system is fine, but the check is too strict. We call this kind of system **non-deterministic** — "the same input does not always give the same output."

Why this needs a mindset shift: you stop asking "is the output identical?" and start asking "does the output have the *properties* a correct answer must have?" A property is something you can check with code, even when the wording changes: it has the right fact, it is valid JSON, it calls the right tool, it is under 100 words, it scores above a quality bar. Without this shift, two bad things happen. Either your tests fail at random and everyone learns to ignore them ("that one is just flaky"), or you stop testing the model part at all. Both leave you with no protection.

How it works: you split the checks into two kinds. Code that does not call a model (chunking, tool math, routing rules) still gets normal exact tests — it *is* always the same. Code that does call a model gets property checks. Setting `temperature=0` (the setting that makes the model pick its most likely words) makes answers *more* stable, but not fully the same across runs or model updates. So treat it as a helper, never as a reason to go back to exact-match checks. For very important cases, run the same input a few times and check a **pass rate** ("passes 5 out of 5" or "at least 4 out of 5"), not one lucky run.

When to use which kind of check:

Situation	What to do	Why	Small example
The code never calls a model (chunking, a tool's math, parsing)	Exact check with `==`	It is always the same, so an exact check is correct and fast	`assert len(chunks) == 3`
The model must pick one label from a fixed list	Exact check on the label only, not on the explanation text	The label is the part your code uses; the wording around it can change	`assert result.category == "billing"`
The model writes free text	Property checks: key facts present, length, forbidden words absent	Many different wordings are all correct	`assert "refund" in answer.lower()`
The model must return structured data	Validate the shape with Pydantic	Wrong shape breaks the next step, wording does not	`Ticket.model_validate_json(raw)`
Quality can't be checked by code ("is this summary good?")	LLM-as-a-judge with written rules (see below)	Code can't judge meaning; a rubric-driven judge can	`assert score >= 4`

Situation	What to do	Why	Small example
A rare but serious case (a wrong refund amount)	Run it several times, check the pass rate	One passing run can be luck	`assert passes >= 4` out of 5

Real-world examples, by situation:

A support-ticket classifier (like Project 1):

```
# Bad: fails when the model says "This is a billing issue." instead
assert reply == "Category: billing"

# Good: check the one property that matters
assert "billing" in reply.lower()
```

A tool-calling agent that must look up an order: don't check what the agent *said*. Check what it *did*: `assert tool_calls[0].name == "get_order"` and `assert tool_calls[0].args["order_id"] == "A123"`. The action is what matters, and it is much more stable than the text.

A JSON extraction step (invoice fields from an email):

```
from pydantic import BaseModel

class Invoice(BaseModel):
    invoice_number: str
    total: float

# raises if the shape is wrong
invoice = Invoice.model_validate_json(raw_output)
# the number itself must be exact
assert invoice.total == 1250.0
```

Here the *shape* is checked by Pydantic, and the one fact that must be exact (the total) still gets `==`.

Where you'll meet it: every exercise in this document, and the Build Task for Project 4. In Doc11 and Project 5, each agent in the pipeline gets its own property checks ("the researcher returns at least 3 sources", "the writer's draft mentions the topic"), because an exact check on a chain of several model calls would almost never pass twice. Doc19 runs these same checks as a release gate before every deploy.

A common mistake: writing `assert response == "expected text"` against a real model call, seeing it pass once, and moving on. Next week it fails with an equally correct answer. Someone

marks it "flaky" and skips it, and now nothing tests that feature. How to spot it: a test that fails on re-run with no code change, where the "wrong" output looks fine when you read it. Fix it by asking "what property made the expected text correct?" and checking that property instead.

Quick cheat sheet:

- No model call inside → exact `==` checks. Model call inside → property checks.
- Check what the system *does* (label, tool call, JSON shape) before what it *says*.
- `temperature=0` makes output more stable, not fully the same — never rely on it for exact matches.
- For important cases, run several times and check a pass rate.
- A test that fails at random is worse than no test — people learn to ignore it.

The testing layers, and what each one actually checks

A real LLM system has many parts: plain Python functions, tools, an API, an agent loop, and the model calls inside. **Testing layers** means you test each part at its own level, from small and fast (one function, no model) up to big and slow (the whole system, like a user sees it).

Why the layers matter: a failure at the unit level points exactly at the broken function. A failure you only see end-to-end tells you *something* is wrong, but not *where* — and every end-to-end run costs real API money and time. So you want each bug caught as low in the list as possible. Without layers, people usually write only end-to-end tests. Those are slow, cost money on every run, fail at random, and when they fail you still have to debug for an hour to find the cause.

How it works: most tests are at the bottom (many, fast, no model). Fewer tests are in the middle (real parts together, model faked). Very few are at the top (real model, full system). To **fake** (or "mock") the model means you replace the real model call with a small function that returns a fixed answer. Then the test is fast, free, and always the same, and it checks *your* code, not the model.

Each layer — what it checks, when you run it, and a small example:

Layer	What it checks	Real model called?	When you run it	Small example
Unit tests	One pure function: chunking, a tool's own math, a routing rule	No	On every save / every commit	`assert chunk_by_paragraph(text) == [...]`
Tool tests	One tool's behavior on its own, with arguments you choose (the model's	No	Every commit	`assert get_order("A123") ["status"] == "shipped"`

Layer	What it checks	Real model called?	When you run it	Small example
	decision to call it is skipped)			
Integration tests	Several parts together (tool call → your validation layer → result), model faked	No (faked)	Every commit	fake model returns a tool call; check the tool ran and the result was saved
API tests	Your FastAPI routes: request/response shape and status codes, not what the agent thinks	Usually no (faked)	Every commit	`client.post("/chat", json={}).status_code == 422`
Agent/workflow tests	The full loop or graph reaches the right final state for given inputs	Yes, or a scripted fake	Before merging a change	"ticket about a refund ends in the `billing` node"
End-to-end tests / evals	The whole system, the way a user uses it, scored on quality	Yes	Before a release, or nightly	run 10+ fixed tasks, judge scores them

Real-world examples, by situation:

Unit test — a text chunker for RAG (Doc08):

```
from chunking import chunk_by_paragraph

def test_chunk_by_paragraph_splits_on_blank_lines():
    text = "First para.\n\nSecond para.\n\nThird para."
    expected = ["First para.", "Second para.", "Third para."]
    assert chunk_by_paragraph(text) == expected
```

Integration test — fake the model, test your own routing code:

```
import triage   # your module; it calls triage.ask_model(...) inside

def fake_ask_model(prompt):
    # a fixed answer: no API call, no cost, same every time
    return '{"category": "billing"}'

def test_refund_ticket_goes_to_billing(monkeypatch):
    # monkeypatch is a pytest helper that swaps a function only for
```

```
    # this test, then puts the real one back
    monkeypatch.setattr(triage, "ask_model", fake_ask_model)
    result = triage.route_ticket("I want my money back")
    assert result.queue == "billing-team"
```

The test is free and always the same. If it fails, the bug is in *your* routing code, not in the model.

API test — a chat endpoint from Doc12:

```
from fastapi.testclient import TestClient
from app import app

client = TestClient(app)

def test_chat_rejects_empty_body():
    response = client.post("/chat", json={})
    # Pydantic validation failed, as expected
    assert response.status_code == 422
```

End-to-end — a scheduled nightly job: a small script calls the real deployed system with 20 fixed questions, scores the answers, and posts the average score to the team chat. Nobody runs it by hand, and it costs a few cents a night.

Where you'll meet it: this document's Basic exercise is the unit layer, the Intermediate exercise is the tool layer, and the Build Task adds an eval layer on top. In Project 4 and Doc11, each agent is a function you can test on its own with a faked model, before you test the full supervisor graph. Doc14 uses the same idea in reverse: when the whole system fails, you go down the layers to find where. Doc19 runs the fast layers on every commit and the slow eval layer before each release.

A common mistake: testing only end-to-end, with the real model, "because that is what users see." What it causes: a slow suite that costs money every run, fails at random, and never tells you which part broke — so people stop running it. How to spot it: your test folder has no test that runs without an API key. Fix it by pulling pure logic out of the model-calling functions, so it can get fast unit tests, and faking the model in integration tests.

Quick cheat sheet:

- Many fast tests at the bottom, few slow tests at the top.
- Unit, tool, integration and API tests should run with no API key and no internet.
- Fake the model when you test *your* code; call the real model only when you test *quality*.
- A failure low in the layers tells you *where*; a failure only at the top tells you *that*.
- Keep model calls in small, separate functions — then they are easy to fake.

Regression testing for prompts

A **regression** is when something that worked before stops working after a change. For LLM systems, the change is often small: a few words in a prompt, a new model version, a different `temperature`, a new tool description. **Regression testing for prompts** means you keep a fixed set of cases with fixed scoring rules, and you run all of them again after *every* such change, then compare the new scores with the old ones.

Why this matters: a prompt change can quietly make things worse on cases you didn't look at while making the change. "Quietly," because the system still runs with no errors — it just answers worse. You fix the one example you were looking at, and you break three others you forgot. Without a fixed suite, you only find out when a user complains, which costs much more than catching it before you ship.

How it works:

1. Pick a fixed set of cases (see the next topic). Include the easy cases, the hard cases, and every bug you have fixed before.
2. Run the suite on the current version and save the result as the **baseline** (the scores you compare against).
3. Make your change (prompt, model, setting).
4. Run the same suite again. Compare each case and the average with the baseline.
5. If the score drops more than a small allowed amount, the change fails — fix it or don't ship it.

Keep the prompt in a file under version control (git), with a version name, so every score is tied to one exact prompt. The same idea is used in Doc19, where prompts and settings are treated as tracked versions.

When to run a regression check, and when it is not needed:

Situation	What to do	Why	Small example
You edited a system prompt	Run the full suite, compare with baseline	Small wording changes can shift behavior on unrelated cases	`pytest evals/ -m regression`
You switched the model (e.g. to a newer or cheaper one)	Run the full suite, and compare cost and speed too	A new model can be better on average but worse on your key cases	baseline 0.91 → new 0.84: stop
You changed a tool description the model reads	Run the agent/tool-choice cases	The model decides which tool to call from that text	check `tool_calls[0].name` per case
A user reported a bad answer	Add that input as a new case, then fix the prompt	The same bug should never come back unnoticed	new case `"refund_in_urdu"`

Situation	What to do	Why	Small example
You only renamed a variable in pure Python code	Normal unit tests are enough	No model behavior changed	`pytest tests/unit`
You changed only the page style of the UI	No prompt regression run needed	The model's input and output did not change	—

Real-world examples, by situation:

A baseline check in a script:

```
import json
from pathlib import Path

# a small drop can be normal noise; a big one is a regression
ALLOWED_DROP = 0.03

def check_regression(new_scores: dict[str, float]) -> None:
    baseline = json.loads(Path("evals/baseline.json").read_text())
    new_avg = sum(new_scores.values()) / len(new_scores)
    old_avg = sum(baseline.values()) / len(baseline)
    worse = []
    for case in new_scores:
        if new_scores[case] < baseline.get(case, 0) - 0.2:
            worse.append(case)
    if new_avg < old_avg - ALLOWED_DROP or len(worse) > 0:
        raise SystemExit(
            f"Regression: avg {old_avg:.2f} -> {new_avg:.2f}, "
            f"worse cases: {worse}"
        )
    print(f"OK: avg {old_avg:.2f} -> {new_avg:.2f}")
```

It checks both the average *and* single cases. A better average can hide one important case that got much worse.

A cost-cutting model switch for a support chatbot: the team moves from a large model to a small, cheaper one. The average score drops only from 0.90 to 0.88, but the "angry customer asks for refund" cases drop from 1.0 to 0.4. The per-case check catches it, and the team keeps the large model just for refund tickets.

A bug report becomes a test case: a RAG bot over company HR documents answered "20 days leave" when the policy says 24. After fixing the retrieval prompt, the team adds `{"question": "How`

`many leave days do I get?", "must_include": ["24"]}` to the suite. That bug can never return silently.

Prove the suite works, on purpose: delete one key instruction from a working prompt and run the suite. If the score does not drop, your rules are too loose. This is this document's Failure exercise.

Where you'll meet it: this document's Failure exercise and Build Task. In Project 5 and Doc19, this suite becomes the gate in CI/CD (the automatic checks that run before code is deployed): no merge if the score drops. In a multi-agent system (Doc11), you keep a small suite *per agent* plus one suite for the whole pipeline — a prompt change in the researcher agent can break the writer agent's output two steps later.

A common mistake: changing the test cases and the prompt at the same time, then comparing with the old baseline. What it causes: the scores are no longer comparable, so a real drop can look like a gain (you removed the hard cases) or a gain can look like a drop. How to spot it: `tasks.py` and the prompt file both changed in the same commit, and the report compares with an old baseline. Fix it by changing one thing at a time, and saving a new baseline on purpose whenever the case list changes.

Quick cheat sheet:

- Every prompt, model, or tool-description change → run the full suite before you ship.
- Save a baseline; compare the average *and* each case.
- Every real bug report becomes a new test case.
- Keep prompts in files under git, so each score belongs to one exact version.
- Make a prompt worse on purpose once — if the score doesn't drop, fix the suite.

Test sets and LLM-as-a-judge

A **test set** (also called an "eval set") is a fixed group of inputs, each with clear rules for what a good answer looks like, run automatically. It is the testing version of a unit-test suite, but for behavior you can't check with `==`. When a rule can't be checked by code (is this summary actually *good*, not just non-empty?), **LLM-as-a-judge** uses a second model call to score the first model's answer against a **rubric** — a short list of written scoring rules you put in the judge's prompt.

Why this matters: without a test set, "it seems better" is just a feeling from the last three answers you read. Without a judge, you can only check simple things like length and keywords, so a fluent but wrong answer passes. **Why the judge's prompt needs the same care as any other prompt:** a badly written judge (for example, one that gives high scores to long answers no matter how correct they are) will happily approve bad answers and give you false confidence. The judge is not a neutral referee. It is a piece of software you built, and it can be wrong like anything else you build.

How a good test set is built:

- Write each case's rules *before* you look at any model output, so you don't just describe whatever the model happened to say.
- Mix easy cases, hard cases, edge cases (empty input, other language, very long input), and cases from real bugs.
- Start small: 10-30 good cases are more useful than 500 careless ones.
- Use code checks first (keywords, JSON shape, tool name), and the judge only for what code can't check.

How a good judge works: give it the question, the answer, and (if you have one) a reference answer. Give it a small score scale (1-5, or pass/fail) with a written meaning for each score. Ask for a short reason *before* the score. Ask for structured output so your code can read the score. Keep the judge's `temperature` at 0. Then **check the judge itself**: score 10-20 answers by hand, and confirm the judge mostly agrees with you. Give it obviously bad answers and confirm it scores them low.

Code check or judge — how to choose:

Situation	What to do	Why	Small example
The rule is a keyword, number, or format	Code check	Free, fast, always the same	`assert "24" in answer`
The rule is "the right tool was called"	Code check on the tool call	The action is exact even when the text isn't	`tool_calls[0].name == "search_docs"`
The rule is about meaning: correct, complete, polite, based on the sources	LLM judge with a rubric	Code can't understand meaning	"5 = all facts correct and nothing invented"
You want to compare two prompt versions	Pairwise judge: "which answer is better, A or B?" — and swap the order and ask again	Judges often prefer whichever answer comes first	run A-vs-B and B-vs-A
High-risk decisions (medical, legal, money)	Judge plus human review of a sample	A judge can be wrong in confident ways	review 10% of judged answers by hand
You have no rubric yet	Write the rubric first, don't start the judge	"Is this good?" gives random, untrustworthy scores	—

Real-world examples, by situation:

A test set for a RAG bot over company documents (`tasks.py`):

```
TASKS = [
    {"id": "leave_days",
     "question": "How many annual leave days do I get?",
     "must_include": ["24"],
     "rubric": "Correct number, cites the HR policy, no invented rules."},
    {"id": "unknown_topic",
     "question": "What is the CEO's home address?",
     "must_include": [],
     "rubric": "Politely refuses; does not invent an address."},
]
```

A judge with structured output (OpenAI SDK, same client as Doc04):

```
from openai import OpenAI
from pydantic import BaseModel, Field

client = OpenAI()

JUDGE_PROMPT_V1 = """You grade answers from a company HR assistant.
Rubric:
5 = every fact is correct and supported by the reference; nothing invented.
3 = mostly correct, but one small fact is missing or unclear.
1 = a wrong fact, or an invented fact.
Length does NOT make an answer better.
Write your reason first, then the score."""

class Judgement(BaseModel):
    reason: str
    score: int = Field(ge=1, le=5)

def judge(question: str, answer: str, reference: str) -> Judgement:
    completion = client.chat.completions.parse(
        model="gpt-4o-mini",
        temperature=0,
        messages=[
            {"role": "system", "content": JUDGE_PROMPT_V1},
            {"role": "user", "content": (
                f"Question: {question}\n"
                f"Reference: {reference}\n"
                f"Answer: {answer}"
            )},
        ],
        response_format=Judgement,
    )
    return completion.choices[0].message.parsed
```

Note the line "Length does NOT make an answer better" — it is there to block the most common judge bias.

Testing the judge itself:

```
def test_judge_scores_wrong_answer_low():
    j = judge(
        "How many leave days?",
        "You get 30 days, plus unlimited sick leave.",
        "24 days",
    )
    assert j.score <= 2, j.reason
```

A writing agent in a content pipeline (Project 4): the judge checks "does the article cover all 3 points from the brief, and is every claim supported by the researcher's notes?" A keyword check can't do this, but a rubric-driven judge can.

Where you'll meet it: this document's Real-world exercise and Build Task (`judge.py`). In Doc11 and Project 4, a "reviewer" or "critic" agent is really an LLM judge *inside* the system — the same rubric rules apply. Project 13 (PR review) is a judge for code. In Doc19, judge scores on live traffic samples are how you notice slow quality decline.

A common mistake: a judge prompt like "Rate this answer from 1 to 10." with no rubric. What it causes: scores that mostly follow length and confident tone, and change from run to run — so a long, wrong answer gets 8/10 and your suite says everything is fine. How to spot it: give the judge a short correct answer and a long wrong one. If the long wrong one scores higher, your judge is broken. Fix it with a written rubric, a small scale, "reason before score", and a check against your own hand scores.

Quick cheat sheet:

- Write the rules before you look at the outputs.
- Code checks first; the judge only for meaning.
- Judge = rubric + small scale + reason before score + `temperature=0` + structured output.
- Test the judge: it must score an obviously bad answer low.
- Save the judge prompt with a version name, like any other prompt.
- For A-vs-B comparisons, swap the order and ask twice.

Watching your system: what to record for each run

Observability means you can understand what your system did from the records it leaves behind, without being there when it ran. For an LLM system, this means you can rebuild "what happened in this one run" after the fact. So for every run you record: the full input, every step and tool call with

what it returned, the final output, timing, and token/cost usage. One run's full record is often called a **trace**, and each step inside it is a **span**.

Why this is different from normal app logging: a normal app usually fails with a crash, and the error log shows where. An LLM system often fails with a *quality* problem: a wrong answer that sounds believable, with no error at all. Without a recorded run, you can't look back and see why one specific bad answer happened — and the model can't reliably explain its own reasoning afterwards. With a record, you can see "the retriever returned the old policy file" or "the agent called `search` 9 times and hit the step limit."

How it works: create one ID per run (a `run_id`) at the start. Pass it to every step, and put it on every log line. Log each step as structured data (JSON), not as a free sentence, so you can search and filter later. Use the same `logging` setup from Doc01 — levels and handlers still apply. Tools like LangSmith (for LangChain/LangGraph) or OpenTelemetry (a standard format many tracing tools accept) do this for you and show traces in a web page, but the idea is the same as the small version below.

What to record for each run:

What to record	Why	Small example
`run_id` (and user or session ID)	Links every step of one run together	`"run_id": "7f3a..."`
Full input, plus prompt version and model name	You can re-run the exact same case later	`"prompt_version": "triage_v3", "model": "gpt-4o-mini"`
Each step: name, input, output, time taken	Shows *where* a run went wrong	`"step": "retrieve", "docs": ["hr_2024.pdf"], "ms": 120`
Each tool call: name, arguments, result or error	Most agent bugs are a wrong tool or wrong arguments	`"tool": "get_order", "args": {"id": "A123"}`
Final output and status (ok / error / step limit)	Tells you what the user actually got	`"status": "ok"`
Tokens and cost per model call, and total time	Finds slow and expensive runs	`"prompt_tokens": 812, "completion_tokens": 95`
User feedback (thumbs up/down) if you have it	Links a real complaint to its exact trace	`"feedback": "down"`

When to record what:

Situation	What to do	Why	Small example
Development on your laptop	Record everything, at `DEBUG` level	You are hunting bugs	full prompts in the log
Production	Record every run's metadata; full text only if allowed	Storage cost and privacy	tokens, time, status, `run_id` always
User data includes personal details (phone, card number, CNIC)	Mask it before logging	Logs are read by many people and kept for a long time	`"phone": "03**-****567"`
A very high-traffic endpoint	Keep full traces for a sample (e.g. 10%) plus all errors	Full traces for every run can cost too much	`if status == "error" or random() < 0.1:`
API keys and secrets	Never record them	Logs leak; see the `.env` rules in Doc01	—

Real-world examples, by situation:

A small tracer for any model call (no extra library):

```
import json
import logging
import time
import uuid

logger = logging.getLogger(__name__)   # same logger setup as Doc01

def log_step(run_id: str, step: str, data: dict) -> None:
    record = {"run_id": run_id, "step": step}
    for key in data:
        record[key] = data[key]
    logger.info(json.dumps(record, default=str))

def answer_question(question: str) -> str:
    run_id = str(uuid.uuid4())
    log_step(run_id, "input", {
        "question": question,
        "prompt_version": "qa_v2",
        "model": "gpt-4o-mini",
    })
    start = time.perf_counter()
    response = client.chat.completions.create(
        model="gpt-4o-mini",
```

```
        messages=[{"role": "user", "content": question}],
    )
    answer = response.choices[0].message.content
    log_step(run_id, "llm_call", {
        "ms": round((time.perf_counter() - start) * 1000),
        "prompt_tokens": response.usage.prompt_tokens,
        "completion_tokens": response.usage.completion_tokens,
    })
    log_step(run_id, "output", {"answer": answer, "status": "ok"})
    return answer
```

A support desk complaint: a customer says "the bot told me my order was cancelled, but it wasn't." You search the logs for that session ID, find the trace, and see the tool `get_order` returned an error, and the model guessed "cancelled." The fix is in the tool error handling (Doc06), not the prompt. Without the trace, you would have edited the prompt and fixed nothing.

A scheduled report job gets expensive: the monthly bill doubles. You group the logs by `step` and add up `prompt_tokens`, and find that one summarise step now sends a whole 80-page file instead of the top 5 chunks. Token counts per step made this a 10-minute fix.

LangChain / LangGraph apps: set the environment variables `LANGSMITH_TRACING=true` and `LANGSMITH_API_KEY` (in `.env`, never in code), and every chain, agent, and tool call is traced for you in LangSmith, with the same kind of data as the table above.

Where you'll meet it: In your Doc12 API, create the `run_id` when a request arrives, and you can save each run's record in the same SQLite database. In Doc14, these records are the first place to look when a bug report comes in, before you try to reproduce it. In Doc11, Project 4 and Project 5, every agent logs with the same `run_id`, so you can follow one request through the supervisor, the researcher and the writer, and see which agent added the wrong fact and how much each agent cost. Doc19 turns these records into dashboards for cost, speed and quality over time. Traces of real bad answers are also the best source of new cases for your regression test set.

A common mistake: logging only the final answer (or only errors), like a normal web app. What it causes: when a believable wrong answer is reported, you have the answer but not the retrieved documents, the tool results, or the prompt version — so you can't tell *why*, and you end up guessing. How to spot it: pick one random past run and try to answer "which tools were called, with what arguments, and what did they return?" If you can't, your records are not enough. A second mistake is the opposite: logging full user messages with personal data and API keys into a file everyone can read.

Quick cheat sheet:

- One `run_id` per run, on every log line of every step and every agent.
- Record input, each step and tool call, output, time, tokens, prompt version, model.
- Log structured JSON, not free sentences.

- Mask personal data; never log secrets.
- Can you rebuild one random past run from the logs? If not, record more.
- Every bad answer you find in the traces becomes a new test case.

Go Deeper (Optional)

You don't need any of these to understand the Core Concepts above — use them if you want a second explanation or more detail.

- pytest documentation — fixtures, parametrize; your test tool from here on.
- OpenAI Evals (GitHub) — a real framework for testing LLMs; read the README and one example.
- LangSmith documentation — run-tracking and testing, specifically for LangChain/LangGraph apps.

Practice Exercises

Setup for this document's practice code: work inside `13_testing_evaluation_observability/` (same venv as before — if it's not active, `source .venv/bin/activate`). New package for this document: `pip install pytest`.

Where your code lives: all of it under `13_testing_evaluation_observability/practice/`, never loose beside this README. Copy in, unchanged: Doc08's `chunking.py`, and Doc06's `tools.py` with the files it imports (`http_client.py`, `exceptions.py`, `logging_setup.py`). Exercises are grouped **by topic, not by difficulty level** — the same convention as Doc01-12 — so one topic's growth from basic to intermediate stays visible in one file.

The full file layout, all exercises:

```
practice/
├── chunking.py, tools.py, ...        copied in, unchanged
├── chunking_unit_test_practice.py    Basic
├── llm_output_testing_practice.py    Real-world + Edge cases
│                                       (2 sections)
├── regression_catch_practice.py      Failure
├── support_prompt.txt                  the prompt Failure breaks
└── tool_test_no_llm_practice.py      Intermediate
```

Why each script exists:

- `chunking_unit_test_practice.py` — learns pytest on code with no model inside, where an exact `==` is right.

- `llm_output_testing_practice.py` — a judge with rules written first, a check that the judge itself can be trusted, and an exact-match test rewritten into a property check.
- `regression_catch_practice.py` — makes `support_prompt.txt` worse on purpose and proves the suite's score drops, naming the tasks that caught it.
- `tool_test_no_llm_practice.py` — tests Doc06's tools directly, success and failure, with no model, no key, no network.

How to run each exercise: if two exercises below are really about the same thing, save them together in ONE script named after that topic, with each level's version as its own clearly labeled section inside it. Run everything from inside `practice/`: test files with `pytest <file> -v`, and `regression_catch_practice.py` with `python`.

For this document:

- Basic (`chunking_unit_test`) is its own topic — save it as `practice/chunking_unit_test_practice.py`.
- Intermediate (`tool_test_no_llm`) is its own topic — save it as `practice/tool_test_no_llm_practice.py`.
- Real-world (`llm_judge_scoring`) and Edge cases (`flaky_test_fix`) are both about testing LLM output that is never exactly the same twice — writing scoring rules, and rewriting an exact-match test into a property check — save them together as `practice/llm_output_testing_practice.py`, with each level as its own section.
- Failure (`regression_catch`) is its own topic — save it as `practice/regression_catch_practice.py`.

Why group by topic instead of by level: if you save each exercise by difficulty level instead, the different versions of the same idea end up scattered across separate files, and you can never see how one topic grows from simple to harder in one place. Grouping by topic keeps that growth visible — open one file, and you see the whole journey for that one thing, from basic to intermediate, side by side.

Jump to an exercise: Basic · Intermediate · Real-world · Edge cases · Failure · Build Task

Basic — a normal, exact test

- **What:** a normal pytest test for `08_rag`'s `chunking.py` (pure logic, no LLM call, exact checks).
- **Why:** this is the foundation layer — get comfortable with ordinary, always-the-same testing before adding the harder, non-deterministic layer on top.
- **When you'll hit this for real:** every pure-logic function in every project — tools' internal math, routing functions, chunking, anything with no model call inside it.
- **How to code it:** `def test_chunk_by_paragraph(): chunks = chunk_by_paragraph(sample_text); assert len(chunks) == 3` — an exact, ordinary `assert`, run with `pytest`.

- **Save as:** `practice/chunking_unit_test_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Intermediate — test a tool without calling the LLM

- **What:** a test for `06_tools_function_calling`'s tool functions, faking the LLM entirely.
- **Why:** this separates "does my tool's own logic work" from "does the model choose to call it" — two completely different questions that should be tested separately.
- **When you'll hit this for real:** any tool with real logic worth testing on its own, independent of whether the model ever calls it correctly.
- **How to code it:** call Doc06's tools directly with hardcoded arguments — `add.invoke({"a": 2, "b": 3})` — skipping the model entirely, and assert on the return value, for both a success and a failure.
- **Save as:** `practice/tool_test_no_llm_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Real-world — score an answer that isn't exact

- **What:** a test for an LLM answer that can't be checked with `==` — scoring rules written first, then the test built.
- **Why:** this is the actual skill this whole document is about — everything before this exercise was preparation for this one.
- **When you'll hit this for real:** this document's own Build Task, testing Project 4's real output quality.
- **How to code it:** write down 2-3 concrete pass/fail rules first (e.g. "mentions the correct category," "under 100 words"), then write a test that checks the real output against those rules, not against one exact string.
- **Save as:** `practice/llm_output_testing_practice.py`, under a `# Real-world` section (this file also holds the Edge cases exercise below, in its own `# Edge cases` section).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Edge cases — fix a flaky test

- **What:** a test that fails randomly because it checks exact LLM wording, rewritten to check a property instead.
- **Why:** a test that fails randomly gets ignored by everyone eventually — "oh, that one's just flaky" — which means it stops protecting you at all.
- **When you'll hit this for real:** the first time you write `assert response == "expected text"` against a real LLM call and it fails on a re-run with an equally correct answer.
- **How to code it:** take a test asserting exact text equality, and rewrite it to assert a property instead — `assert "billing" in response.lower()`, or `assert`

`is_valid_json(response)`.

- **Save as:** `practice/llm_output_testing_practice.py`, under an `# Edge cases` section (this file also holds the Real-world exercise above, in its own `# Real-world` section).
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Failure — prove the suite actually catches a regression

- **What:** make a prompt worse on purpose, and confirm your test suite actually catches the drop. If it doesn't, find the hole.
- **Why:** an eval suite you've never watched fail is a suite you don't actually know works — this is the one test of the tests themselves.
- **When you'll hit this for real:** this document's own Build Task, and every time you'd otherwise just trust a green checkmark without questioning it.
- **How to code it:** temporarily delete a key instruction from a working prompt, re-run your test suite, and confirm the score actually drops. If it doesn't, your test rules are too loose — tighten them until it does.
- **Save as:** `practice/regression_catch_practice.py`.
- **Stuck?** Hint 1 · Hint 2 · Show me the solution

Build Task — Test Suite for Project 4

Stuck on the Build Task? Hint 1 · Hint 2 · Show me the solution **Goal:** a runnable test suite for Project 4 (the multi-agent system) that would actually catch a real regression.

Requirements:

- A fixed set of test tasks (10+) with clear pass/fail or scored rules — written down before you run anything against them.
- At least one normal, always-the-same test layer (tool logic, chunking, routing functions) with exact checks.
- At least one LLM-as-judge test for final answer quality, with the judge's own prompt written down and reviewed (not just "ask GPT if this is good").
- A regression check: run the suite, make one prompt worse on purpose, run it again, confirm the score drops.

Inputs: the fixed set of tasks; the current Project 4 pipeline (and later, a version you've deliberately made worse).

Outputs: a pass/fail or number for each task, and one overall score.

Constraints: the suite must run without you watching it (`pytest` or a script) — no eyeballing needed to get a result.

Where your code lives: `13_testing_evaluation_observability/practice/build_task/` — run everything from inside that folder. Copy Doc08's `chunking.py` and Doc06's `tools.py` (with the files it imports) in first, unchanged.

Suggested files:

```
13_testing_evaluation_observability/practice/build_task/
├── judge.py             score_with_judge(), versioned judge prompt
├── pipeline.py          run_project_4() — the one door into Project 4
├── regression_check.py  sabotage the prompt, prove the score drops
├── run_eval.py          runs every task, prints, saves report.json
├── tasks.py             10 fixed tasks: "expected" or "rules"
├── test_judge.py        the judge must fail obviously bad answers
├── test_unit_layer.py   chunker + tools, no model
└── writer_prompt.txt    the prompt the regression check breaks
```

Why each file exists:

- `judge.py` — `llm_judge_scoring`'s judge, unchanged: typed result, versioned prompt, `temperature=0`.
- `pipeline.py` — one place that calls Project 4, so swapping the stand-in for your real graph touches one file.
- `regression_check.py` — `regression_catch`'s pass bar and per-task list, run against the real suite.
- `run_eval.py` — the runner: cheapest check per task, a reason for each failure, and a saved `report.json`.
- `tasks.py` — the fixed test set, written before anything runs; facts get a free code check, meaning gets judge rules.
- `test_judge.py` — `llm_judge_scoring`'s calibration set, so a judge that approves everything is caught.
- `test_unit_layer.py` — the fast, free layer from `chunking_unit_test` and `tool_test_no_llm`; a broken function fails exactly one test.
- `writer_prompt.txt` — the writer's prompt in a file, read fresh on every call, so the regression check can edit it mid-run (`regression_catch`).

Functions/Components to build:

- `tasks.py` → the fixed list of test tasks with what a good answer looks like
- `judge.py` → `score_with_judge(task_input, output, rules) -> JudgeResult`, with a saved, versioned judge prompt
- `run_eval.py` → `run_eval_suite()`: runs the full suite, prints and saves an overall report
- `regression_check.py` → `run_regression_check()`: baseline, one on-purpose change, compare

Expected Behavior

- The suite runs without you watching, and gives a clear pass/fail plus score report.
- A prompt made worse on purpose causes a real, measurable score drop when you run the suite again.
- Unit-level tests fail fast, and point at the exact spot — not a vague "something's wrong" — when pure logic breaks.

Test Cases

Scenario	Expected
Normal pipeline, full suite	A passing score above your chosen bar
Prompt made worse on purpose, full suite	A real score drop, ideally below the bar
Tool logic broken directly (unit layer)	The one specific unit test fails, pointing at the exact function
Judge given an obviously bad answer	Judge scores it low, doesn't just approve it

Break-It / Debug Preview

- A test that fails randomly because it checks for exact LLM wording.
- A judge prompt that favors long answers no matter how correct they are.
- Full debugging drill in 14_debugging_lab.

Interview Topics Preview

- How you test something with no single correct answer · LLM-as-judge pitfalls · what you'd watch in production to catch a quiet quality drop · unit vs. integration vs. eval testing for agents.

Move On When

You have a runnable test suite for Project 4 that would actually catch a regression if the prompts were sabotaged. Full details: CURRICULUM.md §4.

Stuck? Ask for **Hint 1** or **Hint 2**. Ask for the full solution only if you say **"Show me the solution."**